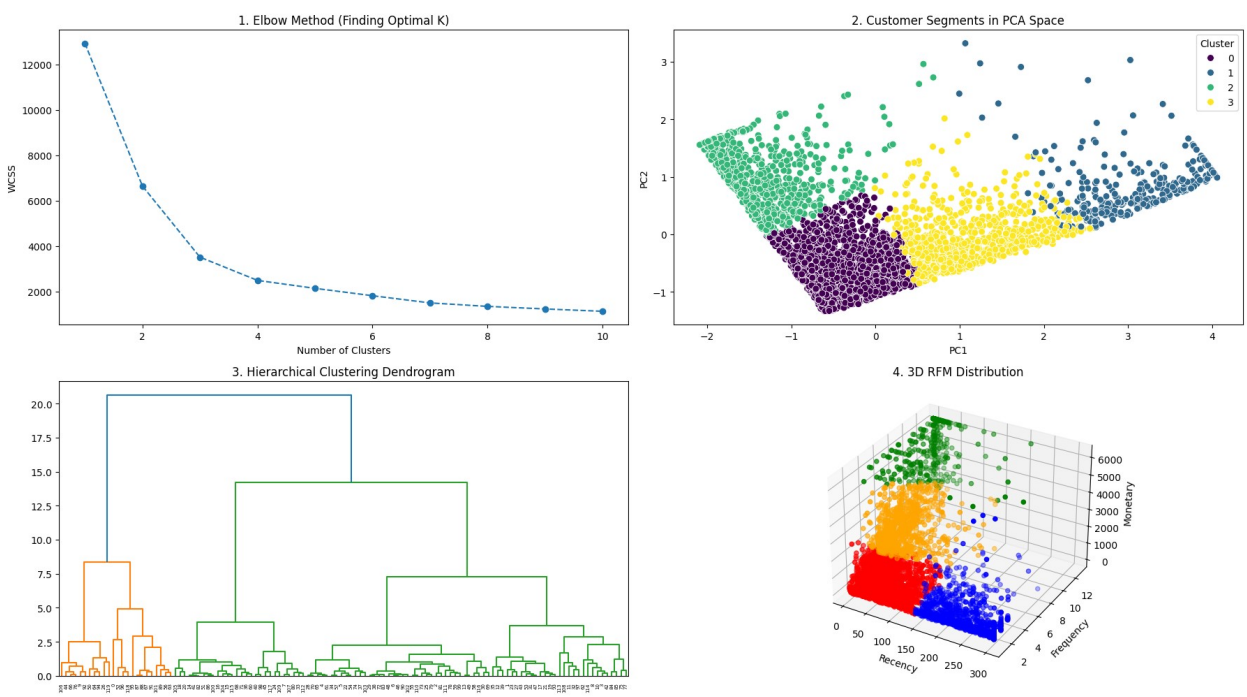
```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
import scipy.cluster.hierarchy as sch
from mpl_toolkits.mplot3d import Axes3D

# ==========================================
# 1. DATA LOADING & CLEANING
# ==========================================
# Using 'latin1' encoding to handle special currency symbols like £
df = pd.read_csv('online_retail_II.csv', encoding='latin1')

# Remove rows without Customer ID and filter for valid transactions
df = df.dropna(subset=['Customer ID'])
df = df[(df['Quantity'] > 0) & (df['Price'] > 0)]

# Set correct data types
df['Customer ID'] = df['Customer ID'].astype(int)
# Robust date parsing
df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'], dayfirst=True)

# ==========================================
# 2. RFM FEATURE ENGINEERING
# ==========================================
# Calculate Monetary value for each row
df['TotalAmount'] = df['Quantity'] * df['Price']

# Reference date for Recency calculation
snapshot_date = df['InvoiceDate'].max() + pd.Timedelta(days=1)

# Group by Customer ID
rfm = df.groupby('Customer ID').agg({
    'InvoiceDate': lambda x: (snapshot_date - x.max()).days, # Recency
    'Invoice': 'nunique',                                    # Frequency
    'TotalAmount': 'sum'                                     # Monetary
}).reset_index()

rfm.rename(columns={
    'InvoiceDate': 'Recency',
    'Invoice': 'Frequency',
    'TotalAmount': 'Monetary'
}, inplace=True)

# Capping outliers at 95th percentile to ensure better cluster
separation
for col in ['Recency', 'Frequency', 'Monetary']:
```

```
    upper_limit = rfm[col].quantile(0.95)
    rfm[col] = np.where(rfm[col] > upper_limit, upper_limit, rfm[col])

# ==========================================
# 3. SCALING & DIMENSIONALITY REDUCTION
# ==========================================
# Standardization is required for K-Means and PCA
scaler = StandardScaler()
rfm_scaled = scaler.fit_transform(rfm[['Recency', 'Frequency',
'Monetary']])

# PCA: Reducing 3D data to 2D for visualization
pca = PCA(n_components=2)
pca_features = pca.fit_transform(rfm_scaled)
pca_df = pd.DataFrame(pca_features, columns=['PC1', 'PC2'])

# ==========================================
# 4. CLUSTERING (K-Means)
# ==========================================
# Find optimal K using the Elbow Method
wcss = []
for i in range(1, 11):
    km = KMeans(n_clusters=i, init='k-means++', random_state=42,
n_init=10)
    km.fit(rfm_scaled)
    wcss.append(km.inertia_)

# Apply K-Means with K=4
kmeans = KMeans(n_clusters=4, init='k-means++', random_state=42,
n_init=10)
rfm['Cluster'] = kmeans.fit_predict(rfm_scaled)
pca_df['Cluster'] = rfm['Cluster']

# ==========================================
# 5. VISUALIZATIONS
# ==========================================
plt.figure(figsize=(18, 10))

# Plot 1: Elbow Method
plt.subplot(2, 2, 1)
plt.plot(range(1, 11), wcss, marker='o', linestyle='--')
plt.title('1. Elbow Method (Finding Optimal K)')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')

# Plot 2: PCA Projection (2D)
plt.subplot(2, 2, 2)
sns.scatterplot(data=pca_df, x='PC1', y='PC2', hue='Cluster',
palette='viridis', s=50)
plt.title('2. Customer Segments in PCA Space')
```

```
# Plot 3: Dendrogram (Hierarchical Clustering - Sampled)
plt.subplot(2, 2, 3)
dendrogram = sch.dendrogram(sch.linkage(rfm_scaled[:120],
method='ward'))
plt.title('3. Hierarchical Clustering Dendrogram')

# Plot 4: 3D RFM Scatter
ax = plt.subplot(2, 2, 4, projection='3d')
colors = ['red', 'green', 'blue', 'orange']
for i in range(4):
    c_data = rfm[rfm['Cluster'] == i]
    ax.scatter(c_data['Recency'], c_data['Frequency'],
c_data['Monetary'], c=colors[i], label=f'C{i}')
ax.set_xlabel('Recency')
ax.set_ylabel('Frequency')
ax.set_zlabel('Monetary')
ax.set_title('4. 3D RFM Distribution')

plt.tight_layout()
plt.show()

# Print summary of the clusters
print("\nCluster Profiling (Average Values):")
print(rfm.groupby('Cluster')[['Recency', 'Frequency',
'Monetary']].mean())
```



```
Cluster Profiling (Average Values):
            Recency  Frequency     Monetary
Cluster
0         51.614044   2.153995   651.446814
1         24.600985  11.192118  5452.792122
2        238.257257   1.537538   464.437832
3         35.805226   6.571259  2244.124715
```